

ELI Capability Catalogue — every action and module, what it actually does

Contents

Headline finding: 208 (183 routable) is real but aliased	2
1. Conversation & reasoning	2
2. OS & application control	2
3. Input & screen control	3
4. Gaze (webcam eye-tracking)	3
5. Media	3
6. Files & documents	4
7. Vision & screen understanding	4
8. Coding & code-repair	4
9. Self-management & self-improvement	4
10. Introspection & audits (grounded, deterministic)	5
11. Memory & identity	5
12. Habits, proactivity & goals	5
13. Self-healing remediation (Linux)	6
14. Voice & transcription	6
15. Time, timers & utility	6
16. Plugins (management)	6
17. Plugin-backed capabilities (the 13)	7
Reading note	7
Part 2 — runtime/ module catalogue (80 files, ~22.6k LOC)	7
Grounding & anti-confabulation	7
Self-honesty / introspection / self-reporting	8
Self-improvement & code-awareness	8
Self-healing remediation	9
Awareness & boot	9
Autonomy / operator (governed)	9
Response surfaces & governance	10
Personal-memory surfaces	10
Typed pipeline plumbing (evidence/packets)	10
Security	11
Part 3 — cognition/ module catalogue (27 files, ~14.0k LOC)	11
Orchestration & agents	11
Inference	12
Reasoning & engagement	12
Context & grounding	12
Persona (the living voice)	13
Output governance (consolidated this session)	13
Profile	13
Part 4 — remaining packages (module catalogue)	14
kernel/ (the engine + boot) — 14.5k LOC	14
core/ (paths, settings, hardware, safety) — 6.6k LOC	14

memory/ — 7.0k LOC	15
perception/ — 6.8k LOC	16
planning/ (autonomy, habits, proactive) — 3.9k LOC	16
coding/ (the frontier coder) — 2.0k LOC	17
learning/ (real LoRA self-training) — 3.3k LOC	17
contracts/, integrations/, system/, utils/, cli/	17
world/ (embodied self-model) — ~1.2k LOC	18
gui/ (PySide6 desktop) — ~20.3k LOC	18
plugins/ — 11 built-ins + manager	19
Part 5 — verified engine request lifecycle (engine.process, line 8219)	19
Part 6 — reasoning modes & autonomous deepening	20

Purpose. A systematic, ground-truth catalogue built by reading the real handlers and modules — not summarised from memory. It exists because conversational summaries of a 126k-LOC project keep undershooting; this is the persisted, exhaustive map. Built in committed batches.

Method. Action list comes from the live `capability_manifest.json` (**208** entries as of 2026-06-18; 183 routable + plugin-backed), verified against the `executor_enhanced.py` dispatch. **The always-current, auto-generated action list with activation phrases is `capabilities_and_actions.md`** — this catalogue is the deeper module-level read. Behaviour grounded in the handlers and the subsystem blueprints. Where a description is inferred from name+structure rather than a line-by-line handler read, it is marked (*inferred*).

Status: Batch 1 — action catalogue (this file will grow with the runtime/ + cognition/ module catalogues and the remaining unread files).

Headline finding: 208 (183 routable) is real but aliased

The manifest’s 208 entries (183 routable) are honest (*measured* by `capability_sync`, not asserted) but inflated by **alias families** — multiple action names routing to one behaviour. Collapsed, there are roughly **~110 distinct capabilities**. Alias families are grouped below so the real surface is visible.

1. Conversation & reasoning

Action(s)	What it does
CHAT	The default — full cognitive pipeline (router→agents→orchestrator→reasoning mode→governed output).
ANSWER, DIRECT_RESPONSE, SAY	Direct/short response surfaces (lighter than CHAT). (<i>inferred for some</i>)
SET_AI_MODE	Set the reasoning mode (Quick / Normal / Advanced / Research / Expert). Legacy names still accepted as input.
SEQUENCE, SEQUENCE_STEP	Multi-step action chaining (run a sequence of actions). (<i>inferred</i>)
TEMPLATE, NOOP	Internal scaffolding / no-op.

2. OS & application control

Action(s)	What it does
OPEN_APP = OPEN_APPLICATION = LAUNCH_APP	Launch an app, resolved via the system_index (your machine's executables — thousands, indexed live) + portable_app_control. On failure → remediation offer (real apt/snap/flatpak install on confirmation).
CLOSE_APP = CLOSE_APPLICATION = EXIT_APP = QUIT_APP	Close/quit an app.
FOCUS_APP, HIDE_APP	Focus / hide a window.
MINIMISE_APP = MINIMIZE_APP, MINIMISE_WINDOW = MINIMIZE_WINDOW, MAXIMISE_WINDOW	Window state control (UK/US spellings aliased).
MINIMISE_ALL, RESTORE_WINDOWS, TILE_WINDOWS, NEXT_WINDOW, PREVIOUS_WINDOW	Desktop window management.
SWITCH_WORKSPACE	Switch virtual desktop/workspace.
OPEN_SYSTEM_SETTINGS, OPEN_AUDIO_SETTINGS, OPEN_POWER_SETTINGS, OPEN_FILE_SYSTEM, OPEN_COMMUNICATION_HUB, OPEN_MEDIA_HUB, OPEN_NETWORK_BROWSER	Open specific OS setting panels / hub launchers.
OPEN_IDE, OPEN_IN_IDE	Open the IDE (optionally at a file).
OPEN_BROWSER, OPEN_URL	Open browser / a URL (local hand-off; respects net toggle for fetches).

3. Input & screen control

Action(s)	What it does
KEYBOARD	Send keypresses (os_controller.press_key).
MOUSE_CONTROL	Move/click the mouse (os_controller.mouse_click).
VOLUME	Get/set system volume.
SET_CLIPBOARD, GET_CLIPBOARD	Read/write the clipboard.
SCREENSHOT	Capture the screen.

4. Gaze (webcam eye-tracking)

Action(s)	What it does
GAZE_ENABLE, GAZE_DISABLE	Start/stop the MediaPipe gaze engine (writes latest_gaze.json @10Hz).
GAZE_CALIBRATE	Run gaze calibration.
GAZE_CLICK	Move the cursor to where you're looking and click.
GAZE_STATUS	Report gaze-engine state.

5. Media

Action(s)	What it does
PLAY_MEDIA	Play a song/playlist on a named platform (Spotify playlist-tab / YouTube Mix-radio); honest about reachability; explicit platform never falls through to a second.
PAUSE_MEDIA, STOP_MEDIA, NEXT_MEDIA, PREVIOUS_MEDIA, REPEAT_MEDIA, SHUFFLE_MEDIA	MPRIS/playerctl transport control.

Action(s)	What it does
MEDIA_CONTROL SKIP_YOUTUBE_AD	Generic media-control dispatch. Skip a YouTube ad. (<i>inferred</i>)

6. Files & documents

Action(s)	What it does
CREATE_FILE, CREATE_FOLDER, READ_FILE, LIST_DIR SUMMARIZE_FILE	Filesystem ops (path-allowlist gated). Summarise a file's contents (also the route for "read your persona.auto.txt / settings").
CONVERT_DOCUMENT CREATE_DOCUMENT = CREATE_DOC = DOC_GENERATE = GENERATE_DOCUMENT = WRITE_DOCUMENT	Convert document formats. Generate a document (the Report-Builder family).
ANALYZE_CSV, ANALYZE_PDF, ANALYZE_PDF_FOLDER, ANALYZE_IMAGE, OCR_IMAGE	Local file-type analysers (CSV stats, PDF text/structure, image VL description, OCR).

7. Vision & screen understanding

Action(s)	What it does
ANALYZE_IMAGE	Local GGUF vision-language description of an image.
OCR_IMAGE	Tesseract OCR text extraction.
AMBIENT_VISION	Toggle periodic background screen glances.
SCREEN_LOCATE	OCR-locate a named UI element on screen ("the button that says X").
SCREEN_READ_ANALYZE	Screenshot → analyse what's on screen.
IMAGE_STATUS	Report vision/image-engine state.

8. Coding & code-repair

Action(s)	What it does
CODE_SOLVE	The frontier coding agent: plan → DAG decompose → UCB tree search → verify (syntax/exec/tests) → repair → bug-memory.
GENERATE_SCRIPT = CREATE_SCRIPT = WRITE_SCRIPT = GENERATE_CODE = WRITE_CODE	Generate a script — routes through the coding agent (falls back to inline gen).
GENERATE_PROJECT	Generate a multi-file project.
EXAMINE_CODE	Tiered file scan (syntax/import → lint → gated LLM logic review) → offer fix.
FIX_FILE, CONFIRM_CODE_FIX, CANCEL_CODE_FIX	Apply/confirm/cancel a verified auto-reverting code fix.
FILE_AUDIT, SHOW_DIFF, CODE_CHANGES	Audit a file / show a diff / report recent code changes.

9. Self-management & self-improvement

Action(s)	What it does
SELF_REPORT	Grounded runtime self-report (deterministic, from live state).
SELF_ANALYZE, SELF_IMPROVE, SELF_IMPROVEMENT_LOG	Analyse failures / surface improvement proposals / show the improvement log.
SELF_PATCH	Generate+apply a verified self-patch.
SELF_TEST	Run self-tests.
SELF_UPGRADE = SELF_UPDATE	Maintenance: git pull, pip, rebuild FAISS/KG, refresh manifest + system index.

10. Introspection & audits (grounded, deterministic)

Action(s)	What it does
RUNTIME_STATUS, RUNTIME_AUDIT	Live runtime report; RUNTIME_AUDIT also runs health probes (plugin mgr, memory, agent bus, habit integrity, recent failures).
GUI_RUNTIME_AUDIT, IMPORT_AUDIT, DIAGNOSE_WRAPPERS, RESOLVE_RUNTIME_PATHS FRONTIER_STATUS, ELI_IDENTITY_AUDIT	GUI/runtime audits, import smoke-audit, executor-wrapper diagnostic, path resolution. Full cross-system matrix / identity-classification audit.
COGNITION_STATUS, EXPLAIN_COGNITION_RUNTIME, EXPLAIN_MEMORY_RUNTIME EXPLAIN_ALL_REASONING_MODES, REASONING_MODE_STATUS	Cognition + memory runtime explainers (verbatim, deterministic).
EXPLAIN_LAST_RESPONSE, EXPLAIN_LAST_FAILURE, NAME_SOURCE_AUDIT, ROUTING_FAULT_EXPLAIN	Describe the 5 reasoning modes / current mode.
HARDWARE_PROFILE, GPU_STATUS, GET_STATUS, AWARENESS_STATUS	Explain the last answer/failure, the source of your name, or a routing fault.
HELP, LIST_CAPABILITIES	Hardware/GPU/general/awareness status.
	Help / list the capability surface.

11. Memory & identity

Action(s)	What it does
MEMORY_RECALL, MEMORY_STORE, MEMORY_STATS, MEMORY_STATUS	Recall/store memories; memory inventory/stats.
PERSONAL_MEMORY_SUMMARY, PERSONAL_MEMORY_DEEP_EXPLAIN	“What do you know about me” (clean) / deep memory-internals explain.
USER_IDENTITY_SUMMARY, USER_INFO_REPORT, REFRESH_USER_INFO	Who-you-are summary / full profile report / rebuild the living profile.
SET_USER_NAME	Set the user’s name.
MESSAGE_TIME_QUERY	“What time did I first/last message (today)” — from conversation_turns.
CLEAR_CHAT_HISTORY	Clear chat history.
PERSONA_LOCK_SET, PERSONA_LOCK_CLEAR, PERSONA_LOCK_STATUS	Lock/unlock/inspect the persona.

12. Habits, proactivity & goals

Action(s)	What it does
CONFIRM_HABIT, DECLINE_HABIT, HABIT_STATUS MORNING_REPORT	Approve/decline a detected habit; report habits. The consolidated morning brief (news digest + activity + attention).
PROACTIVE_START, PROACTIVE_STOP, PROACTIVE_STATUS EXECUTE_GOAL BACKGROUND_JOBS, CHECK_JOB	Control the proactive daemon. Execute a mission-layer goal (governed). List background tasks / check a job id.

13. Self-healing remediation (Linux)

Action(s)	What it does
PREPARE_REMEDIATION	Diagnose a failure + build a repair plan (e.g. install a missing app via apt/snap/flatpak).
CONFIRM_PENDING_REMEDIATION, CANCEL_PENDING_REMEDIATION CHECK_TARGET_STATUS	Execute / cancel the pending repair (sudo terminal, lock-handling, verification). Check whether a target app/path is now present.

14. Voice & transcription

Action(s)	What it does
DICTATE, TRANSCRIBE LISTEN_FOR_COMMAND STT_DIAGNOSTICS, VOICE_DIAGNOSTICS	Dictation / transcribe audio (faster-whisper). One-shot listen. Diagnose the STT/voice pipeline (mic, ducking, wake gate).
SAY / SPEAK (<i>plugin: tts</i>)	Speak text (Piper TTS).

15. Time, timers & utility

Action(s)	What it does
TIME = GET_TIME, DATE = GET_DATE SET_TIMER, SET_ALARM CHECK_CHRONAL_ALIGNMENT	Current time / date. Timers/alarms. (<i>inferred</i>) Easter-egg: “Chronal alignment nominal. Local time: ...” (a playful time report).
DATA_FABRICATOR	Generate a document from a topic (via CREATE_DOCUMENT) and open it in an editor (code/gedit/kate/nano).
RUN_CMD, SHELL_EXEC	Run a shell command — fail-closed (blocked unless ELI_ALLOWED_CMDS/ELI_FULL_CONTROL).

16. Plugins (management)

Action(s)	What it does
PLUGIN_LIST, PLUGIN_SEARCH, PLUGIN_STATUS PLUGIN_INSTALL, PLUGIN_UNINSTALL, PLUGIN_ENABLE, PLUGIN_DISABLE	List installed / search registry / status. Install/remove/enable/disable a plugin.

17. Plugin-backed capabilities (the 13)

Action	Plugin	What it does
WEB_SEARCH	web	DuckDuckGo/SearXNG web search (toggle-gated).
GET_WEATHER	weather	Local geocode + open-meteo (toggle-gated).
NEW_NOTE/WRITE_NOTE, LIST_NOTES, SEARCH_NOTES	notes	Markdown notes with FTS.
ADD_EVENT, LIST_EVENTS	calendar	ICS calendar events.
POMODORO_START, POMODORO_STOP, POMODORO_STATUS	pomodoro	Focus timers.
CPU_USAGE, RAM_USAGE, SYSTEM_STATS	system_stats	CPU/RAM/disk/network.
SMART_HOME	smart_home	Smart-home control via ELI's own MQTT device server (ESPHome / Tasmota / Zigbee2MQTT); rooms, scenes, real automations. Home Assistant removed.
SPEAK	tts	Text-to-speech.
NEWS_FETCH	(executor + news tool)	Fetch + synthesise news (rolling 3-hourly reflections → 24h digest).

Reading note

Alias families that collapse the count: app-open (3), app-close (4), minimise (4 across window/app + UK/US), document-generate (5), script/code-generate (5), time/date (2 each), self-upgrade/update (2). Removing pure aliases yields **~110 genuinely distinct capabilities** — still an unusually broad surface for a local single-user assistant, spanning OS control, media, files, vision, gaze, voice, coding, memory, introspection, autonomy, remediation, and plugins.

Part 2 — runtime/ module catalogue (80 files, ~22.6k LOC)

The largest package. It's the **grounding/governance + introspection + plumbing** layer that wraps the probabilistic model. Grouped by function:

Grounding & anti-confabulation

Module	LOC	Role
deterministic_grounding_gate.py	292	Renders control/status answers directly from live runtime (bypasses the model). 7 stacked render_action layers + an immutable policy engine (the active chain; one dead v10 fragment removed this session).

Module	LOC	Role
grounding_escalation.py	453	When a checkable factual question is poorly grounded by the bus, escalates through agent tiers instead of letting the model confabulate (the “Eminem’s real name” failure class). Regexes that catch vague/dynamic status confabulation (“currently processing updates...” and image-status fabrication). Control-action evidence contract: build evidence → validate the model’s output doesn’t violate it → finalise. Runtime-status question detection, live-evidence build, repair/validate. Collects memory evidence for grounding a turn. Gates what gets stored — refuses to persist internal dumps / error-pattern noise as memory.
diagnostic_patterns.py	112	
control_contracts.py	943	
contracts/grounded_control.py,(in contracts/)		
contracts/runtime_status.py		
memory_evidence.py	221	
persistence_gate.py	188	

Self-honesty / introspection / self-reporting

Module	LOC	Role
live_introspection.py	730	Live runtime snapshot, last trace, stored user name, mines user-fact candidates, agents-for-action, build_report. The engine’s live diagnostic dispatcher (handle_diagnostic_action) — deterministic answers for RUNTIME_STATUS / EXPLAIN_* / IMPORT_AUDIT.
deterministic_introspection.py	573	
truth_report.py	301	Runtime truth report (git, nvidia, GGUF runtime, import health). Full cross-system status matrix (run-time/memory/awareness/proactive/image/identity-classification audit (source inventory, contract counts, capability matrix). Current reasoning-mode reporting. Inventory of experimental projects.
frontier_status.py	417	
eli_identity_audit.py	416	
reasoning_status.py	201	
experimental_inventory.py	146	

Self-improvement & code-awareness

Module	LOC	Role
self_improvement.py	1206	Failure logging/clustering, patch generate→verify→apply→ auto-revert , full patch cycle, plugin-stub gen.
code_examiner.py	717	Tiered file error scan (syntax/import → lint → gated LLM) → offer → verified fix.
code_monitor.py	262	Detects source changes via git diff, classifies by subsystem, summarises for memory/context (ELI is aware of its own code changes).
capability_sync.py	387	AST-discovers the live capability surface, diffs, writes capability_manifest.json — this is why the count is <i>measured</i> , not asserted.
generated_script_guard.py	1072	Validates LLM-generated scripts, quarantines invalid ones , and ships vetted canned scripts for known patterns (GPU-watch, redshift, etc.).
repair_policy.py	23	Policy line: proposal layers vs source-mutation layers.

Self-healing remediation

Module	LOC	Role
grounded_remediation.py	1604	Diagnoses failures (missing app/path/browser) → builds an apt/snap/flatpak repair plan → offers → executes (sudo terminal, lock-handling, verify). Writes incident records.
incident_log.py	21	

Awareness & boot

Module	LOC	Role
awareness_boot.py	282	Boots all awareness subsystems at startup, returns an AwarenessState the engine queries.
action_commitment.py	168	Detects when ELI's reply COMMITTS to an action (so the pipeline re-runs and actually does it — no fake actions).

Autonomy / operator (governed)

Module	LOC	Role
operator_state.py, operator_feed.py, operator_feed_normalized.py pending_proposal.py	~220 115	Operator console state: proposals, goals, self-model status, event feed. Pending-proposal state (extract/set/clear).
approval_engine.py	103	Who may propose / evaluate a proposal record (governance).

Response surfaces & governance

Module	LOC	Role
user_visible_response_surface.py	326	Installs the engine's user-visible response surface (runtime/identity/name-source formatting + streaming coercion).
visible_output.py, visible_text.py, output_sanitizer.py	~200	Central visible-output contract; stringify/sanitise streamed output.
response_policy.py, response_contracts.py, response_packets.py	~205	Classify response mode; per-action contracts; final-answer request packets.
final_response_assembly.py, final_response_provider.py, fastpath_responder.py	~175	Assemble the final prompt; per-action generation decoration; fastpath context.

Personal-memory surfaces

Module	LOC	Role
personal_memory_surface.py	431	"Is this a personal-memory query" + surface builder.
personal_memory_clean_response.py	304	Clean "what do you know about me" report (reset-aware, poison-filtered, dynamic-fact aging).
personal_memory_deep_response.py	343	Deep memory-internals explain (schema/tables/functions) + routing-fault explain.
profile_extractor.py	740	Extracts user facts from turns (role/interests/field/"remember that I..."), writes user_patterns + LLM session summaries; recency refresh.
identity_validation.py, identity_guard.py	~240	Validate identity candidates; persona/identity lock state.

Typed pipeline plumbing (evidence/packets)

Module	LOC	Role
evidence_ledger.py	394	Records artifacts/events with signatures; recent generated artifacts; status evidence.
evidence_arbitration.py	195	Scores competing evidence (stage packets + tool results + goals), dedup-by-fingerprint, keep-max.
evidence_store.py, stage_packet_store.py, stage_packets.py, pipeline_models.py, retrieval_packets.py, typed_stage_bridge.py, packet_native_downstream.py, single_pass_authority.py, tool_result_*	~700	The typed packet substrate: route/plan/evidence/generation/output packets flow between stages; this is the plumbing behind “no fake actions”.
background_tasks.py	253	In-process multi-threaded task manager (heavy work → job id → CHECK_JOB).
runtime_policy.py	76	Per-turn budgets/timeouts/context size from runtime snapshot.

Security

Module	LOC	Role
security.py	207	SecurityManager — fail-closed command/path/app allowlist sandbox.

Part 3 — cognition/ module catalogue (27 files, ~14.0k LOC)

The thinking layer: agents, orchestration, inference, persona, reasoning, governance.

Orchestration & agents

Module	LOC	Role
agent_bus.py	3138	14 specialist agents on a dependency DAG (topological layers) + calibrated weight-free confidence aggregation + per-action agent selection.
orchestrator.py	928	The 12-stage retrieval pipeline: HyDE → FAISS + FTS5 + KG-BFS + RAG → hybrid merge → heuristic rerank (lexical × recency × importance; cross-encoder is the designed upgrade) → context assembly.

Module	LOC	Role
hyde.py	69	Hypothetical-document-embedding query expansion. Candidate reranking (token overlap + source priority). Wraps introspection for the bus (pipeline/memory/runtime/audit). LLM intent parsing fallback (GGUF, cached).
reranker.py	75	
introspection_agent.py	173	
llm_intent.py	148	

Inference

Module	LOC	Role
gguf_inference.py	2677	Model-agnostic GGUF inference: model resolution (no baked model), family-aware chat templating, graceful GPU-layer fallback, streaming, output cleaning, token budgeting. Thin GGUF broker (infer) used by agents/coding/patching. Chat response + streaming helpers, turn persistence.
inference_broker.py	200	
chat_model.py	292	

Reasoning & engagement

Module	LOC	Role
reasoning_modes.py	631	The 5 modes — canonicalisation, per-mode private system instruction, execution contract (samples/branches/stages + dynamic token budget), reasoning-leak stripping. Session depth tracking → auto-escalates reasoning mode (Quick→Normal→Advanced→Research) as a conversation deepens; session narrative. Turn-scoped pinned facts (pin/absorb/evict/persist/restore).
engagement_tracker.py	249	
working_memory.py	325	

Context & grounding

Module	LOC	Role
context_synthesiser.py	567	Builds the precise prompt context: persona handoff, turns block, vector block, live-runtime brief, budgeting. Lighter persona+memory context builder + fallback guard.
context_builder.py	118	

Module	LOC	Role
grounded_status.py	644	Identity + memory-inventory rendered directly from profile/DBs (direct grounded answers).

Persona (the living voice)

Module	LOC	Role
persona.py	364	Canonical persona authority — base + auto sections, preferences, compose/refresh.
persona_updater.py	737	Re-derives the persona overlay from memory/reflection/habits/runtime patterns; KG population; stale-fact aging.
persona_hygiene.py	131	Cleans/dedups/prunes the auto-persona.
persona_status.py, persona_values.py	~108	Persona status report; values store.

Output governance (consolidated this session)

Module	LOC	Role
output_governor.py	1224	Canonical governance: sanitize, role-prefix/identity-drift repair, self↔user confusion repair, style cleanup, confabulation detection, quality scoring, memory-worthiness, GGUF-artifact cleaning (clean_gguf_artifacts), evidence validation.
response_governance.py, response_sanitizer.py	28+14	Re-export shims → output_governor (kept for back-compat).
tone_analyzer.py	367	Analyses recent user turns → tone preferences ELI adapts to over time.

Profile

Module	LOC	Role
user_info_builder.py	746	The living, versioned user profile: multi-source gather, noise filter, categorise, hash, diff-on-change.

Part 4 — remaining packages (module catalogue)

kernel/ (the engine + boot) — 14.5k LOC

Module	LOC	Role
engine.py	13374	CognitiveEngine — the conductor: persona, generation settings, the 5 _run_* reasoning passes, grounding overrides, synthesis prompt build + the context-bloat cap, fragment/placeholder guards, dispatch gate to bus vs orchestrator, startup loops (reflection/habit/scheduler/self-improve/proactive).
world_model.py	252	Symbolic self-model: Identity/Runtime/Memory/Goal/Capability states + snapshot/merge.
state.py	330	User/runtime state + profile (active user id, name, profile text).
self_upgrade.py	233	Self-upgrade orchestrator (git pull, pip, rebuild FAISS/KG, manifest, system index).
scheduler.py	74	Kernel thread-pool/timer scheduler (generic).
pipeline.py, task_bus.py	~180	Pipeline step description; task bus.

core/ (paths, settings, hardware, safety) — 6.6k LOC

Module	LOC	Role
runtime_settings.py	1096	Canonical settings.json load/save, legacy-key migration, portable-path healing/sanitising.
hardware_profile.py	1040	Auto-detect GPU/VRAM/RAM → fit ctx/gpu_layers/batch (KV-cache + compute-buffer reserve); model discovery; hardware authority enforcement.
paths.py	551	Single-source-of-truth path resolution (data/config/cache/db/models/voices...).
startup_hardware_optimizer.py	599	Boot-time hardware optimiser (GPU select, layer/ctx allocation, mode presets).
config.py	343	Thin config shim over runtime_settings (canonical key mapping).
model_download.py	544	Curated GGUF catalogue + VRAM-based recommend + download.

Module	LOC	Role
dag.py	474	Generic DAG engine (Kahn topo-order, parallel layers, critical path) — shared by the agent bus + coding engine. Per-boot runtime budget derivation. Offline-by-default socket-level network gate (fail-closed) + guarded_urlopen/http_get_json. Factory-reset of memory/identity (scrub names, clear DBs, backup). User-tunable knowledge-gathering limits + synthesis cap registry (GUI-surfaced). is_grounded_query classifier. STT-robust self-harm detector + persona steering directive. path helpers, first-run, compat, ownership map.
dynamic_runtime_budget.py	215	
netguard.py	204	
memory_reset.py	218	
cognition_tunables.py	237	
grounding.py	147	
crisis_guard.py	111	
portable_paths.py, db_paths.py, legacy_paths.py, first_run*.py, compatibility.py, architecture_contracts.py	small	

memory/ — 7.0k LOC

Module	LOC	Role
memory.py	4461	The Memory god-class (~50 methods): semantic store/recall, conversations, habits, failures/improvements, weight decay, KG bridge, mark_failure_resolved, disable_invalid_habit_rules. Entity/relation graph + multi-hop BFS (related) + context_for_prompt + extract-from-memory. Habit rules/events store + cheap embed/recall. FAISS index (L2, 1/(1+dist) sim) + nomic embedder + keyword fallback + auto-rebuild. OS app/exe/dir index (the launcher backing — your machine's executables, thousands, indexed live per machine). inspection/compat/session helpers.
knowledge_graph.py	564	
habits_memory_db.py	470	
vector_store.py	411	
system_index.py	238	
memory_truth.py, memory_adapter.py, memory_service.py, stores.py, sqlite_memory.py, populate_memories.py	small	

perception/ — 6.8k LOC

Module	LOC	Role
audio_stt.py	1848	faster-whisper STT + VoiceGate (wake-word, debounce, incomplete-command wait), self-echo suppression, output ducking, per-user voice profile bias.
vision.py	692	Local GGUF VL (Moondream fast / primary), hot-swap with text model, CPU-pinned CLIP, OCR.
tts_router.py	667	Piper/espeak TTS, voice selection, unspeakable-fragment guard.
os_controller.py	573	Screenshot/volume/keys/mouse/clipboard + gaze_click.
screen_locator.py	389	OCR (tesseract) → locate a named UI element on screen.
gaze_engine.py	298	MediaPipe face-gaze + calibration mapper + One-Euro smoothing → latest_gaze.json @10Hz.
local_whisper_stt.py, ambient_vision.py, ana- lyze_{pdfs,image,mesh,csv}.py, log_rotation.py, voice_worker*.py	~1.2k	Whisper backend; ambient glances; file analysers; log rotation; voice workers.

planning/ (autonomy, habits, proactive) — 3.9k LOC

Module	LOC	Role
proactive_daemon.py	1301	Background 10-min loop: pattern/code analysis, habit detect+offer, morning report, error tracking.
autonomy_scheduler.py	212	Policy-gated (observe/proposal/goal-driven) goal scheduler w/ cooldown + attention queue.
habits.py	344	Habit detection (timestamp→HH:MM clustering), offer/pending state, disabled-by-default.
habits_scheduler.py	152	Fires active habits at their time (self-heals legacy rows, once-per-minute dedupe).
goal_store.py, goal_models.py, goal_tick.py, operator_goal_actions.py	~400	Mission goals (priority/cadence/risk/constraints/success-criteria) → governed proposals.
attention_queue.py, proposal_queue.py, proposal_*.py, jobqueue*.py, autonomy_controller.py, task_planner.py	~900	Attention ranking; proposal queue/archive; job queue; safe autonomy ticks.

coding/ (the frontier coder) — 2.0k LOC

Module	LOC	Role
agent.py	215	CodeAgent.solve — composes the loop; broker-backed generator; full provenance.
bug_memory.py	340	Bug classification + (signature→fix) long-term SQLite memory w/ fuzzy recall.
verification.py	310	Gate ladder (syntax→exec→tests) + weighted scoring + test synthesis.
sandbox.py	204	Bounded isolated execution (scrubbed env, CPU limit, timeout).
search.py	176	UCB1 tree search over a temperature-ladder beam.
plan_graph.py	164	Subtask DAG decompose + topological solve + compose.
planner.py, cost.py	~210	Planner/implementer split; cost → foreground/background decision.

learning/ (real LoRA self-training) — 3.3k LOC

Module	LOC	Role
lora_trainer.py	646	Real torch/PEFT/transformers Trainer LoRA run.
lora_trainer_guard.py	561	TrainerTarget plans + safety validation.
dataset_builder.py	528	Build supervised dataset from logged turns/corrections (PII-redacted, deduped).
lora_eval.py	481	Adapter eval suite.
bootstrap_phi3_base.py, base_model_resolver.py, dataset_filters.py, export_trainable_dataset.py, merge_reviewed_datasets.py, training_preflight.py	~1.4k	Trainable base download/resolve, quality gates, export/merge, preflight.

contracts/, integrations/, system/, utils/, cli/

Module	LOC	Role
contracts/runtime_status.py	471	Canonical runtime-status evidence contract (detect question, build live evidence, validate/repair).
contracts/grounded_control.py	210	Grounded-control synthesis guard (evidence-complete? suppress bad clarifications).

Module	LOC	Role
integrations/mpri2/playerctl_backend.py	635	MPRIS2/playerctl media backend (player resolve, transport, status).
integrations/ollama/client.py	334	Optional Ollama backend client.
system/portable_app_control.py	338	Cross-platform installed-app discovery + open/close/minimise.
utils/platform_compat.py	804	Cross-platform layer (open url/file/app, volume, clipboard, notify, executables).
utils/log.py, cli/headless.py	~200	Central logger; headless terminal REPL.

world/ (embodied self-model) — ~1.2k LOC

Module	LOC	Role
agency/autonomy_engine.py	258	Awareness vector (easing/decay) → avatar-room routing + symbolic-object creation during reasoning; provenance/snapshots/journal.
agency/{policy,goal_ecology,habit_engine,reflection_bridge,permission_classes,goal.py		Remission/instability; default habits; reflection→world; world identity.
core/schemas.py, core/ontology.py	~160	World state schemas (rooms/objects/events/actions/awareness); object templates.
renderers/pyside6/{world_scene,world_panel}.py	405	The PySide6 “Eli’s World” tab (house/rooms/avatar/objects, live state).
avatar/*, persistence/*, world_event_bus.py, local_world_bridge.py	~700	Avatar behaviour/locomotion/persona-map; journal/provenance/snapshots/storage; event bus + bridge.

gui/ (PySide6 desktop) — ~20.3k LOC

Module	LOC	Role
eli_pro_audio_gui_MKI.py	10992	Main window: 12 tabs + adapters (CentralMemory/LocalModel/Ollama/Executor bridges, the _GUIEngineAdapter), chat, drag-drop, reasoning-mode auto-select, all toggles.

Module	LOC	Role
labs_tab.py	5696	Labs workspace: Notebook, Memory browser, Jupyter launcher, Calculator(+constants), Physics tables, Report Builder (evidence-grounded docs), File-Chat, Workspaces, Sim-IDE. Launcher / first-boot auto-tune / main().
app.py	742	
panels/startup.py	737	
panels/settings.py	672	Advanced Settings (Agents/Models/ Cognition /Plugins/Self-Upgrade).
docks/operator_console_dock.py	803	Governed operator console (proposals/goals/policy/attention).
widgets/, tabs/, panels/agent_wizard.py, docks/proactive_dock.py, qt_compat.py	~1k	Ollama selector, experimental/world tabs, agent wizard, proactive dock, Qt shim.

plugins/ — 11 built-ins + manager

Module	Role
manager.py (553L)	Discover/install/enable/disable/execute; auto-load; builtin-stub gen; registry.
web, web_automation, weather, calendar, notes, pomodoro, smart_home, system_stats, media, tts, document_reader	The 11 built-in plugins (see Part 1 §17).
base/base.py	Plugin base class + action validation + loader.

Part 5 — verified engine request lifecycle (engine.process, line 8219)

Read end-to-end this session. The real entry flow, in order:

1. **Pipeline trace id** assigned (ELI_PIPELINE_TRACE for observability).
2. **Prompt-injection guard FIRST** — `_eli_sanitize_user_input` runs before the router or LLM ever sees the raw text, so injected instructions can't reach them unsanitised. (A security control that sits at the very top of the pipe.)
3. **Multi-question splitter** (ELI_MULTI_QUESTION_SPLITTER_V2) — if a message has >1 ? **and** >25 words, it splits into standalone sub-questions (cap 4) and answers each; a second pass splits single-? compounds like “who are you, and who am I”. So genuinely multi-part asks get multiple grounded answers.
4. Router → agent bus / orchestrator gate → reasoning mode → executor → governed output (Parts 1-4).

Note: engine.py carries **51 PHASE... markers** — the same “added beside, not folded in” accretion as the grounding gate. It works and is well-guarded, but the interior is layered patches; that residue (engine 13.4k / executor 14.3k / GUI 11.0k) is documented here at behavioural level, not line-by-line.

Part 6 — reasoning modes & autonomous deepening

The 5 reasoning modes were renamed (public layer; internal keys unchanged) and made adaptive + self-deepening.

Public mode	Internal key	Multi-pass algorithm	Agent time budget	Deepen iters	Confidence target
Quick	quick	single pass	1.0×	0 (never)	0.45
Normal	chain_of_thought	one reasoned pass	1.0×	1	0.55
Advanced	self_consistency	N samples + consensus	1.5×	2	0.65
Research	tree_of_thoughts	branch + prune	2.0×	3	0.75
Expert	constitutional_ai	draft → critique → revise	2.5×	4	0.80

Auto-selection (cognition/engagement_tracker.py): the mode is escalated automatically as a conversation deepens (Quick→Normal→Advanced→Research) without the user asking. (Internal strategy keys are stable; users only ever see the public names.)

Per-mode agent time budgets (reasoning_modes.mode_budget_multiplier, Stage 1b): each mode scales every agent's timeout — quick/normal unchanged, deeper modes get proportionally more time. Threaded via intent["_mode_budget_mult"] → agent_bus._collect_layer. Tunable in the Cognition tab (cog.mode_budget_*).

Confidence-driven iterative deepening (runtime/grounding_escalation.escalate, Stage 2): on a poorly-grounded *checkable-factual* turn, ELI re-dispatches the agent bus broadly, **escalating the reasoning mode one tier per iteration** and (Stage 3a) **raising the gather budget** each pass (1.5×→2.0×→2.5×), until grounding crosses the per-mode target or the per-mode iteration budget is spent (early no-improvement break). Banter/opinion/command/meta never deepen.

Background deepening (runtime/background_deepening.py, Stage 3b): a Quick answer returns instantly; if it was a poorly-grounded factual turn, a background thread keeps gathering (broad re-dispatch at Advanced + 2× gather) and surfaces a better answer in the Proactive panel **only if** it crosses grounding 0.55 and isn't degenerate. Tightly gated (quick-only, grounding<0.40, factual-only, deduped, 1 in-flight, 10-min cooldown). Toggle: cog.background_deepen / ELI_BACKGROUND_DEEPEN.